

Génération de graphes aléatoires à l'aide de méthodes Monte-Carlo par chaînes de Markov

Félix Lopez, Nicolas Alam

26 juin 2025

1 Introduction

Les graphes aléatoires sont des objets mathématiques essentiels pour modéliser des réseaux dans de nombreux domaines. Leur génération permet d'obtenir des comparatifs pour étudier des phénomènes concrets, comme le routage de paquets dans un réseau de machines ou la diffusion de maladies dans un groupe de personnes.

Par exemple, dans le cadre de l'étude d'un protocole de communication entre machines, on peut chercher à évaluer sa robustesse sur différentes structures de réseaux. Générer des graphes aléatoires permet alors de simuler plusieurs organisations possibles des connexions entre les machines, tout en gardant certaines propriétés fixes comme les degrés des sommets. Cela permet de tester si le protocole se comporte toujours correctement ou efficacement sur des réseaux réalistes.

Nous cherchons ici à produire des graphes aléatoires de référence, sans biais. Pour cela le processus d'échantillonnage doit être uniformément aléatoire parmi l'ensemble des graphes conformes à des contraintes données.

Lorsqu'on souhaite générer un graphe aléatoire, il faut d'abord choisir un modèle qui précise les propriétés structurelles que l'on veut respecter. Tous les graphes considérés ici seront supposés simples et non-orientés. Un modèle simple et très étudié est le modèle d'Erdős-Rényi [1], qui regroupe les graphes dont toutes les paires de sommets ont une même probabilité p , indépendamment des autres, d'être reliées par une arête. Malgré le fait que ce choix de contrainte faciliterait la génération, il ne nous permettrait pas de conserver les degrés de notre graphe de départ, une partie essentielle de sa structure. Quand on regarde la distribution de degrés, on remarque que le caractère Poissonien des graphes d'Erdős-Rényi s'éloigne des lois de puissance qui interviennent dans de nombreux graphes réels comme des réseaux sociaux ou des réseaux biologiques.

Pour nos tirages, on utilisera donc le *configuration model* [2, 3], à savoir, les graphes simples dont la séquence des degrés est fixée.

Nous commençons par présenter la méthode classique d'appariement des demi-arêtes, dont les limites justifient le choix d'un algorithme MCMC.

Puis, nous expliquons et énonçons les propriétés nécessaires au fonctionnement de cette seconde approche.

Enfin, nous implémentons et analysons ces deux algorithmes de génération, en suivant pour la méthode MCMC, le protocole expérimental proposé par U. Dutta et A. Clauset

dans leur article "*Convergence criteria for sampling random graphs with specified degree sequences*" (*Journal of Computational and Graphical Statistics*, 2021) [4].

Notations importantes

On note $G = (V, E)$ un graphe, $V = \{v_1, \dots, v_n\}$ l'ensemble de ses sommets, $E = \{\{v_a, v_b\}, \dots, \{v_k, v_l\}\}$ l'ensemble de ses arêtes (ici non orientées), $n = |V|$, $m = |E|$. La séquence de degrés d'un graphe est la liste $\{d_1, \dots, d_n\}$ des degrés de chaque sommet, elle est dite graphique si il existe un graphe simple la réalisant.

2 Approches théoriques

L'échantillonnage d'un graphe du *configuration model* est un problème difficile, il n'existe pas à ce jour, de méthode connue permettant de produire de manière uniformément aléatoire et en temps polynomial, un graphe réalisant une séquence de degrés quelconque. Nous implémentons ici deux approches : la première repose sur l'appariement aléatoire des demi-arêtes (ou *stubs*), méthode classique de construction dans ce modèle, et la seconde est une méthode basée sur les chaînes de Markov Monte-Carlo (MCMC), qui modifie progressivement le graphe obtenu tout en préservant la séquence de degrés.

2.1 Méthode constructive du *configuration model*

Dans cette première approche, on part seulement d'une séquence de degrés graphique $d_1, \dots, d_n$ et d'un graphe vide $G = (V, E)$ où $V = \{v_1, \dots, v_n\}$ et $E = \emptyset$ que l'on va petit à petit peupler d'arêtes.

On commence par créer une liste L où chaque sommet $v_i \in V$ apparaît d_i fois, ce qui correspond à chaque demi-arêtes dont v_i est l'extrémité. On remarque que L contient $2m$ éléments.

Puis on tire à la suite des paires de demi-arêtes u, v que l'on enlève de la liste L avant d'ajouter $\{u, v\}$ à E , sauf si $u = v$ ou $\{u, v\}$ y est déjà, auquel cas on annule l'opération, vide le graphe de toute arête, et on recommence le processus de zéro.

Nous détaillons la procédure complète dans l'algorithme 1.

2.1.1 Problème des distributions de degrés hétérogènes

Ce tirage est uniforme dans le *configuration model* des multi-graphes à boucles or on ne veut générer ici que des graphes simples : sans boucles ni multi-liens. On pourrait naïvement les supprimer après l'appariement mais cela entraînerait plusieurs problèmes :

- La séquence de degrés serait modifiée.
- On perdrait l'uniformité de l'échantillonnage : les sommets de plus fort degrés sont plus susceptibles d'apporter des boucles et multi-liens.

Pour pallier à cela, on itère l'exécution en recommençant à zéro si une boucle ou un multi-lien est détecté, jusqu'à obtenir un graphe simple. L'uniformité est bien préservée car l'ensemble des graphes simples est compris dans celui des multi-graphes à boucles.

Malgré tout, on remarque que la probabilité d'échec de l'algorithme 1 est très élevée lorsque la séquence de degré est hétérogène, ce qui est très fréquent dans l'étude de réseaux réels, rendant cette méthode inutilisable.

Cela justifie le recours à une méthode alternative, décrite dans la section suivante. Les performances de l'exécution répétée de l'algorithme 1 pour générer des graphes simples seront étudiées dans la partie expérimentale du rapport.

2.2 Algorithme MCMC

Dans cette seconde approche, nous ne construisons pas un graphe à partir de zéro comme dans la méthode d'appariement classique (algorithme 1), mais nous partons d'un graphe simple réalisant la séquence de degrés imposée, et nous le modifions progressivement. L'objectif est d'approcher un échantillonnage aléatoire aussi uniforme que possible. Pour cela on utilise un algorithme basé sur les méthodes Monte-Carlo par chaîne de Markov, qui procède par répétition de modifications aléatoires sur le graphe jusqu'à atteindre l'uniformité recherchée.

À chaque itération, on tire uniformément, deux arêtes du graphe, puis on les croise, en échangeant deux de leurs extrémités (2-*swap*, figure 1). Si cet échange crée une boucle ou un multi-lien, on l'annule et on passe à l'itération suivante.

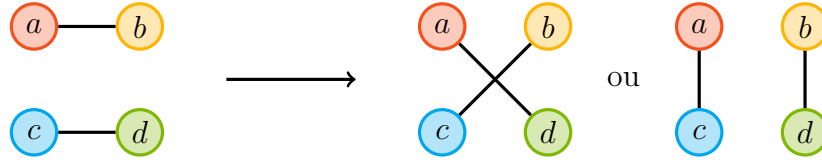


FIGURE 1 – exemple des issues d'un 2-*swap*

On note G_0 le graphe de gauche et $E_0 = \{\{a, b\}, \{c, d\}\}$ l'ensemble d'arêtes associées.

Soit G_1 le graphe résultant d'un 2-*swap* sur G_0 . E_1 peut valoir $\{\{a, d\}, \{b, c\}\}$ ou $\{\{a, c\}, \{b, d\}\}$, les deux seules permutations possibles, à probabilités égales.

2.2.1 Définitions et notations utiles

Avec $G = (V, E)$ le graphe simple de départ.

— **Le k -*swap***

On tire uniformément k arêtes toutes différentes $\{\{x_1, y_1\}, \dots, \{x_k, y_k\}\} \subset E$, ainsi qu'une permutation $\sigma \in S_k$.

Et on remplace dans E , les arêtes tirées par $\{\{x_1, y_{\sigma(1)}\}, \dots, \{x_k, y_{\sigma(k)}\}\}$.

Exemple d'un 2-*swap* dans la figure 1.

— **Le nombre de triangles**

Par triangle on entend : un triplet $\{v_1, v_2, v_3\}$ de sommets distincts, tous reliés entre eux, c'est à dire $\{\{v_1, v_2\}, \{v_2, v_3\}, \{v_1, v_3\}\} \subset E$

2.2.2 Propriétés de la chaîne

L'algorithme MCMC s'appuie sur une chaîne de Markov que nous construisons ici sur l'espace des graphes du *configuration model*.

États : Chaque état correspond à un graphe $G_t = (V, E_t)$ où G_0 est le graphe de départ. Tous sont simples et réalisent la séquence de degrés imposée. Il n'en existe donc qu'un nombre fini.

Transitions : À chaque itération, on choisit uniformément deux arêtes différentes : $\{\{u_1, v_1\}, \{u_2, v_2\}\} \subset E$ que l'on les remplace par une permutation de leurs extrémités : soit $\{\{u_1, v_2\}, \{u_2, v_1\}\}$ soit $\{\{u_1, u_2\}, \{v_1, v_2\}\}$ à probabilité égale. Si la permutation choisie génère une boucle ou un multi-lien, on l'annule. Il est important de remarquer que cet échange que l'on appelle ici : *2-swap* (figure 1), préserve la séquence de degrés.

La convergence de cette chaîne vers une distribution stationnaire unique et indépendante de l'état initial, est garantie par les trois propriétés suivantes.

Irréductibilité : À condition que la séquence de degrés imposée soit graphique, chaque graphe simple la réalisant doit être accessible depuis un autre par une suite de *2-swap*.

Ce resultat est démontré par Fosdick et al. [5, Prop. 4.1].

Aperiodicité : Il n'existe pas d'entier p tel que pour tout t , $G_t = G_{t+p}$. En effet, chaque état G_t , il existe une probabilité non nulle de tirer deux arêtes dont le *2-swap* formerait une boucle ou un multi-lien, c'est à dire de rester dans l'état $G_t = G_{t+1}$,

Positive-Réurrence : L'espace concerné ne contient qu'un nombre fini de graphes simples, or la chaîne de Markov est irréductible, elle est donc automatiquement récurrente-positive.

Nous devons maintenant nous pencher sur une propriété supplémentaire : la symétrie, afin de montrer que cette distribution stationnaire est bien uniforme.

Symétrie - Soient G et G' deux graphes simples réalisant la séquence de degrés. On note $P_{G,G'}$ la probabilité de transition de G à G' .

- si $G \neq G'$ et si il existe un *2-swap* les reliant de la sorte : $G \xrightarrow{2\text{-swap}} G'$ alors on remarque que le choix du *2-swap* est symétrique, tout comme celui de la permutation, car le même *2-swap* appliqué à G' donne G et est tiré avec la même probabilité. Et donc $P_{G,G'} = P_{G',G}$.
- si $G \neq G'$ et si il n'existe pas de *2-swap* les reliant alors $P_{G,G'} = 0 = P_{G',G}$
- si $G = G'$, c'est à dire si le *2-swap* échoue, alors $P_{G,G'} = P_{G',G}$

Réversibilité - La chaîne étudiée admet une distribution stationnaire, est définie sur ensemble fini d'états et est à transitions symétriques, on en déduit qu'elle est aussi réversible. Sa distribution stationnaire est donc bien uniforme sur les graphes du *configuration model*.

Ayant établi l'existence d'un état stationnaire vers lequel la chaîne converge, il est désormais nécessaire d'implémenter un critère d'arrêt, chargé d'évaluer la convergence et de mettre fin au processus de *swap* une fois l'état stationnaire atteint.

2.2.3 Mesure structurelle

La mise en place d'un critère d'arrêt repose sur le choix d'une mesure structurelle, non triviale, représentative de l'évolution de la chaîne. Cette mesure, nous sert à prélever des échantillons à intervalles réguliers sur les graphes de la chaîne. En les analysant via un test statistique, on peut détecter la convergence de la chaîne une fois l'état stationnaire atteint, et à mettre fin au processus de *2-swap*. Pour des raisons pratiques, on favorise les mesures structurelles aux meilleurs temps de calcul, qui peuvent idéalement être réajustées localement après un *2-swap*, nous évitant un re-calcul complet.

On décide de suivre dans un premier temps l'évolution du nombre de triangles du graphe (sous-sous-section 2.2.1), initialisé avec l'algorithme 2 et ajusté après un *2-swap* avec l'algorithme 3.

3 Implémentation et analyse

1

3.1 Méthode constructive

On affirme plus tôt que l'hétérogénéité des degrés d'un graphe est source de boucles et multi-liens lorsque l'on exécute l'algorithme 1. Pour vérifier cette hypothèse, on met en place un test en traitant deux graphes aux densités très proches. On entend par densité : la probabilité p que deux sommets distincts issus d'un tirage aléatoire uniforme soient reliés par une arête.

- G_1 , aux degrés hétérogènes, de densité $p = 3.975 \times 10^{-3}$.
- G_2 , un graphe d'Erdős-Rényi, de densité $p = 3.962 \times 10^{-3}$.

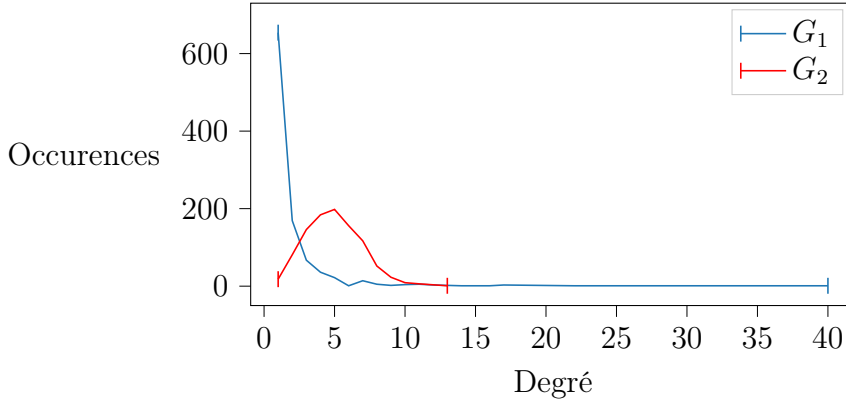


FIGURE 2 – Distribution des degrés de deux graphes : G_1 et G_2

Ce résultat empirique (figure 3) vient appuyer l'hypothèse : les sommets dont le degré est significativement supérieur à la moyenne ont une forte probabilité de générer des défauts par la méthode d'appariement. La génération sans échec de graphes du *configuration model* aux séquences de degrés très hétérogènes nécessite donc un grand nombre de tentatives.

1. Le code source complet est disponible sur GitHub [6]

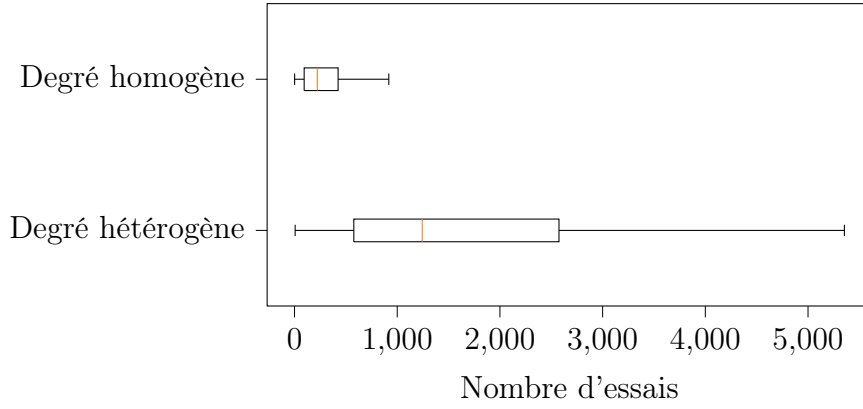


FIGURE 3 – Nombre d’essais de 1000 exécutions réussies

3.2 Méthode MCMC

Dans cette section, nous évaluons empiriquement la convergence de la chaîne de Markov (sous-section 2.2) dans le but d’obtenir un nombre de *2-swap* propre à chaque graphe, à partir duquel on estime avoir atteint l’état stationnaire de la chaîne. Une fois dépassé, on peut considérer le graphe modifié itérativement comme tiré uniformément depuis le *configuration model*. Nous suivons pour cela le protocole expérimental proposé par Dutta et Clauset [4], qui consiste à faire tourner plusieurs chaînes en parallèle, créant à chaque pas un échantillon de graphes : une mesure statistique qui nous sert à évaluer la convergence de la chaîne.

On note :

- G le graphe de départ
- $G_{k,t}$ le $t^{\text{ième}}$ état de la $k^{\text{ième}}$ chaîne
- Avec $k \leq K$, le nombre de chaînes parallèles.
- $m_{k,t}$ une mesure structurelle prélevée sur le graphe de l’état $G_{k,t}$

3.2.1 Protocole

Le principe est le suivant :

- On lance K chaînes de Markov ayant toutes comme état initial : $G_{k,0} = G$.
- On prélève à intervalles de temps réguliers, une mesure structurelle sur l’ensemble des K chaînes.
- On examine l’évolution de ces mesures à l’aide d’un test statistique, et détermine si il est temps ou non de mettre fin à l’exécution.

3.2.2 Choix du *sampling gap*

La détection de la convergence consiste à déterminer si les fluctuations de la mesure structurelle prélevée sur les états visités témoignent d’une distribution stationnaire. Compte tenu de cela, les fortes corrélations entre les états consécutifs de la chaîne posent un problème. En effet, deux graphes séparés par un unique *2-swap*, sont identiques, à l’exception de 4 arêtes. Pour pallier à cela, on suit la démarche de Dutta et Clauset [4] et

on prélève nos mesures à intervalles réguliers, assez espacés pour garantir l'indépendance statistique nécessaire.

Le *sampling gap* ou "écart d'échantillonnage", noté η_0 , désigne le nombre d'itérations entre deux mesures successives, et est dans notre implémentation, commun aux N chaînes parallèles. L'écart η_0 que le *sampling gap* représente doit être assez grand pour que la corrélation entre deux états qu'il sépare G_t et $G_{t+\eta_0}$ soit négligeable : les mesures faites sur une chaîne avec un *sampling gap* trop petit, pourraient être redondantes et ne déclencher l'arrêt du programme que très tard. De même, un *sampling gap* trop grand gâcherait du temps de calcul, en poursuivant aussi le processus sur une chaîne ayant déjà atteint l'état stationnaire.

Ce paramètre doit donc être choisi avec précaution, pour cela, on utilise l'arbre de décision [4, Figure 6] qui a été construit expérimentalement pour générer des graphes à séquence de degré fixée à l'aide d'une procédure de recherche automatique.

3.2.3 Détection de la convergence

Pour détecter empiriquement la convergence de la chaîne de Markov, nous utilisons un test similaire à celui proposé par Dutta et Clauset [4].

Celui-ci consiste à d'abord rassembler les mesures prélevés sur les K échantillons au temps T (au pas $T\eta_0$) dans une liste $L_T = \{ m_{1,T\eta_0}, m_{2,T\eta_0}, \dots, m_{K,T\eta_0} \}$ de taille K .

Puis à appliquer le test de Kolmogorov–Smirnov non-paramétrique à deux échantillons [7] sur les listes L_{T-1} et L_T afin de déterminer si elles peuvent raisonnablement être considérées comme issues de la même distribution.

Si le test de Kolmogorov–Smirnov rejette l'hypothèse nulle selon laquelle les deux échantillons L_{T-1} et L_T proviennent de la même distribution, on réitère le processus de *swap* sur toutes les chaînes parallèles jusqu'au temps $(T+1)\eta_0$, avant d'appliquer le test de Kolmogorov–Smirnov aux deux échantillons L_T et L_{T+1} .

Cette opération est répétée jusqu'à ce que le test ne puisse plus rejeter l'hypothèse nulle, ce que l'on interprète comme un signe de convergence des chaînes vers leurs distributions stationnaires, à savoir la loi uniforme sur l'ensemble des graphes réalisant la séquence de degrés fixée. Sur n'importe quelle chaîne parallèle, chaque pas η_0 supplémentaire à partir de ce point est alors considéré comme un tirage indépendant et identiquement distribué depuis le *configuration model*.

3.2.4 Résultats d'expérience

Nous utilisons pour notre expérience, un graphe provenant de la base de données publique Konect [8], qui représente des collaborations entre musiciens [9].

$n = 198$	$m = 2742$	$\eta_0 = 1214$	Nombre de triangles = 17899
-----------	------------	-----------------	-----------------------------

On initialise et lance ensuite l'exécution de 100 chaînes de Markov ayant ce graphe pour état initial. La figure 4 montre l'évolution de la distribution du nombre de triangles échantillonné sur l'ensemble des chaînes, aux pas : $0.2m$, $0.4m$, $0.6m$, $3.5m$ et $100m$ 2-swaps. On constate que la distribution commence par s'éloigner progressivement de la valeur initiale du nombre de triangles, pour se concentrer autour d'une autre que l'on soupçonne être son espérance sur l'ensemble des graphes réalisant cette séquence de degrés. Cet éloignement général témoigne bien d'une perte de corrélation avec le graphe initial, l'effet recherché.

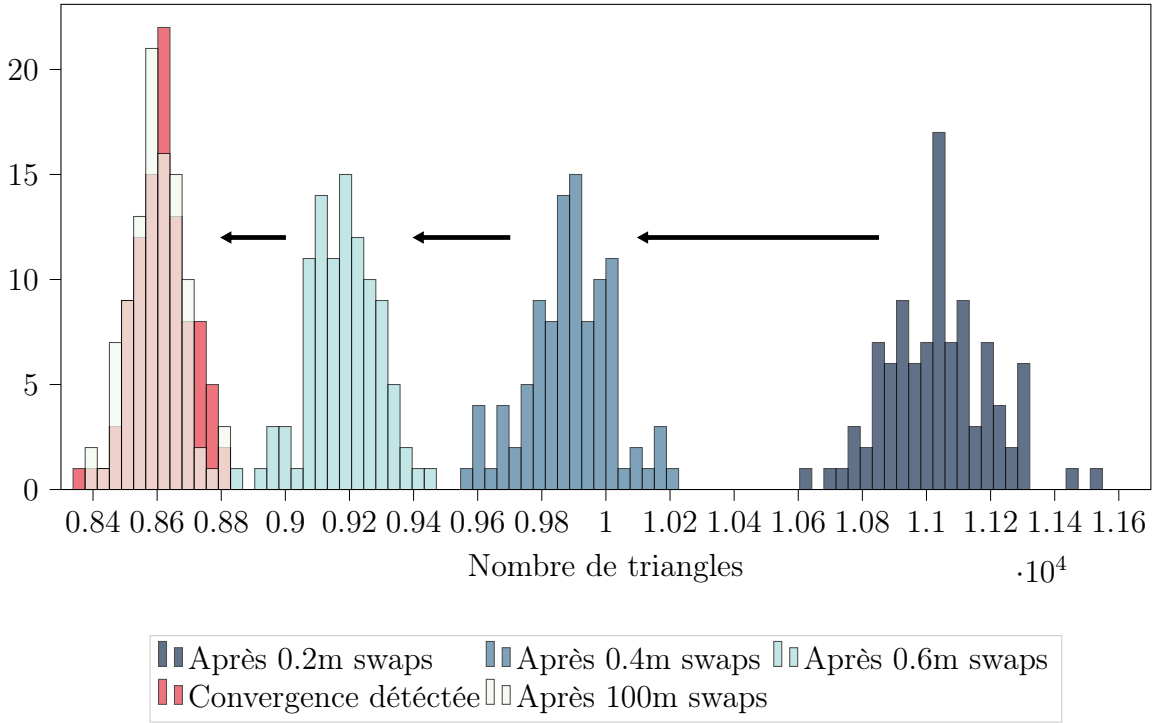


FIGURE 4 – Évolution de la distribution du nombre de triangles dans 100 chaînes de Markov parallèles

3.3 Travail en cours - Contrainte supplémentaire

Après avoir travaillé sur la génération de graphes du *configuration model*, nous nous sommes intéressés à une version plus contraignante du problème en fixant le nombre de triangles des graphes, en plus de leur séquence de degrés. Malheureusement, nous n'avons pas pu atteindre l'objectif initialement fixé pour cette partie : reproduire toute la démarche expérimentale de génération MCMC.

En voici les fondement théoriques, adaptés à cette contrainte supplémentaire :

Conserver un motif, et en particulier toutes ses occurrences permet de préserver des propriétés structurelles du graphe d'origine avec un contrôle beaucoup plus fin qu'en fixant juste la séquence de degrés. On pourrait par exemple dans le cadre d'une analyse de réseaux réels très approfondies, vouloir que l'agencement de certains clusters, reste inchangé dans les modèles de références générés. La solution que l'on présente ici correspond à une version allégée de ce problème car le motif choisi reste très petit.

3.3.1 Problèmes posés par la nouvelle contrainte

Comme précédemment, pour garantir l'existence d'un unique état stationnaire uniforme, on doit s'assurer que la chaîne de Markov définie sur ce nouvel espace vérifie bien les quatre propriétés d'irréductibilité, de symétrie, d'apériodicité, et de positive-réurrence. Ces deux dernières se démontrent de la même façon.

Cependant, lorsqu'on impose un nombre de triangles fixé, certains motifs présents dans le graphe pourraient bloquer toute tentative de 2-swap et invalider ainsi l'irréductibilité de la chaîne. Par exemple, si le graphe contient plusieurs composantes connexes dont un triangle isolé, tout 2-swap permutant une des arêtes du triangles avec celle d'une autre composante décrémenterait le nombre de triangles de 1, annulant la transition. Ce triangle ne sera donc jamais altéré par un 2-swap .

3.3.2 Construction d'une nouvelle chaîne

Pour pallier à ce problème, on propose d'utiliser comme transition un $k\text{-swap}$ légèrement différent, à k variable, proposé par L. Tabourier et J. Karadayi dans leur article "*Probabilistic $k\text{-swap}$ method for uniform graph generation beyond the configuration model*" (*Journal of Complex Networks*, 2024) [10].

Soit $G_t = (V, E_t)$ le graphe d'état au temps t , à chaque itération on tire un entier k tel que $2 \leq k \leq m$, selon la loi suivante : $P(k) \sim k^{-\gamma}$ de paramètre $\gamma \in]1, +\infty[$, puis on réalise un $k\text{-swap}$. Comme précédemment, si le nombre de triangles reste inchangé, on conserve la modification, sinon on la rejette.

Dans le cas précédent (sous-sous-section 2.2.2), l'irréductibilité reposait sur l'existence d'une longue suite de transitions, ce qui n'est plus le cas ici. En effet, la chaîne munie de $k\text{-swap}$ peut désormais atteindre n'importe quel graphe G' du *configuration model* en un seul pas $G \xrightarrow[k\text{-swap}]{} G'$. (voir démonstration dans [10, Section 3.1]). Ce résultat assure donc l'ergodicité de la chaîne, quelques soient les contraintes imposées en plus de la séquence de degrés.

La symétrie de cette nouvelle chaîne est aussi conservée.

Il suffit de prendre $G = (V, E)$ un graphe de départ, $2 \leq k \leq |E|$, $\mathcal{E}_k \subset E$ le nombre et l'ensemble d'arêtes à permuter et $\sigma \in S_k$ une permutation. On note G' le graphe résultant du $k\text{-swap}$ sur G que l'on suppose sans échec. Il existe un $k\text{-swap}^{-1}$ inverse sur G' avec $\sigma^{-1} \in S_k$ et $\mathcal{E}'_k \subset E'$ l'ensemble des arêtes de $\mathcal{E}_k$ après permutation. Comme précédemment, $P_{G,G'} = P(G \xrightarrow[k\text{-swap}]{} G') = P(G' \xrightarrow[k\text{-swap}^{-1}]{} G) = P_{G',G}$

Quant à la mesure structurelle, ne pouvant plus utiliser le nombre de triangles désormais fixe, on suggère d’étudier l’assortativité : coefficient de corrélation (au sens de Pearson) entre le degré d’un sommet et le degré de ses voisins. L’ajustement après un k -swap de cette valeur a pour avantage de s’implémenter en temps quasi-constant : $O(k)$.

Conclusion

Ce projet avait pour but d’implémenter et de comparer des méthodes de génération uniformément aléatoires de graphes du *configuration model*.

Nous avons d’abord évalué empiriquement les limites de la méthode constructive classique.

Puis, nous avons implémenté un algorithme Monte-Carlo par Chaînes de Markov en suivant une démarche issue de la littérature [4].

Enfin, nous avons amorcé une extension qui généralise cette méthode à tout jeu de contraintes supplémentaire à la séquence de degrés [10]. Dans notre exemple : le nombre de triangles. L’intégration de cette contrainte présentée théoriquement n’a pas pu être implémentée par faute de temps pour finaliser les calculs.

Travaux restants Reproduire le protocole expérimental de la sous-section 3.2, étendre cette contrainte à d’autres motifs plus complexes : carrés, cycles élémentaires quelconques

Références

- [1] P. Erdős and A. Rényi. On random graphs i. *Publicationes Mathematicae Debrecen*, 6 :290–297, 1959.
- [2] E. A. Bender and E. R. Canfield. The asymptotic number of labeled graphs with given degree sequences. *Journal of Combinatorial Theory, Series A*, 24(3) :296–307, 1978.
- [3] Béla Bollobás. A probabilistic proof of an asymptotic formula for the number of labelled regular graphs. *European Journal of Combinatorics*, 1(4) :311–316, 1980.
- [4] Upasana Dutta and Aaron Clauset. Convergence criteria for sampling random graphs with specified degree sequences. *CoRR*, abs/2105.12120, 2021.
- [5] Bailey K. Fosdick, Daniel B. Larremore, Joel Nishimura, and Johan Ugander. Configuring random graph models with fixed degree sequences, 2017.
- [6] Félix Lopez and Alam Nicolas. random-graphs-mcmc — code source. <https://github.com/proj-IN013/random-graphs-mcmc>, 2025. commit 18921f6, consulté le 26-06-2025.
- [7] Frank J. Massey Jr. and. The kolmogorov-smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253) :68–78, 1951.
- [8] Jérôme Kunegis. KONECT – The Koblenz Network Collection. In *Proc. Int. Conf. on World Wide Web Companion*, pages 1343–1350, 2013.
- [9] PABLO M. GLEISER and LEON DANON. Community structure in jazz. *Advances in Complex Systems*, 06(04) :565–573, December 2003.

- [10] Lionel Tabourier and Julien Karadayi. Probabilistic k-swap method for uniform graph generation beyond the configuration model. *Journal of Complex Networks*, 12(1), February 2024.

A Algorithmes

A.1 méthode classique de construction du *configuration model* (sous-section 2.1)

Algorithm 1: Tentative de formation aléatoire de paires de demi-arêtes sans boucle ni multi-lien

```

input :  $H$  : tableau de  $2n$  demi-arêtes (entiers)
output:  $H$  trié en  $n$  paires sans boucle ni multi-lien, ou NULL en cas d'échec
1 begin
2   for  $i \leftarrow 0$  to  $n - 1$  do
3      $j_1 \leftarrow$  entier aléatoire dans  $\llbracket 0, 2n - 2i - 1 \rrbracket$ ;
4      $j_2 \leftarrow$  entier aléatoire dans  $\llbracket 0, 2n - 2i - 2 \rrbracket$ ;
5      $pos_1 \leftarrow 2n - 2i - 1$  ; // emplacement pour la première demi-arête
6      $pos_2 \leftarrow 2n - 2i - 2$  ; // emplacement pour la seconde
7     échanger  $H[j_1]$  avec  $H[pos_1]$ ;
8     échanger  $H[j_2]$  avec  $H[pos_2]$ ;
9      $u \leftarrow H[pos_2], v \leftarrow H[pos_1]$  ; // nouvelle paire  $(u, v)$ 
10    if  $u = v$  then
11      return NULL ; // boucle détectée
12    end
13    // vérifie si  $(u, v)$  ou  $(v, u)$  a déjà été créé
14    for  $k \leftarrow 0$  to  $i - 1$  do
15       $a \leftarrow H[2k], b \leftarrow H[2k + 1]$ ;
16      if  $(a = u \wedge b = v)$  ou  $(a = v \wedge b = u)$  then
17        return NULL ; // multi-lien détecté
18      end
19    end
20    return  $H$  ; // tableau de paires valides
21 end

```

A.2 Comptage de triangles

Algorithm 2: Comptage des triangles

Entrée: Un graphe non orienté $G = (V, E)$ représenté par des listes d'adjacence

Sortie : Le nombre total de triangles dans G

```
1  $nb\_triangles \leftarrow 0$ ;  
2 for  $a \leftarrow 0$  to  $n - 1$  do  
3   foreach  $b$  voisin de  $a$  où  $b > a$  do  
4     foreach  $c$  voisin de  $b$  où  $c > b$  do  
5       if  $c$  est voisin de  $a$  then  
6          $nb\_triangles \leftarrow nb\_triangles + 1$ ;  
7       end  
8     end  
9   end  
10 end
```

Algorithm 3: Mise à jour incrémentale du nombre de triangles après un 2-*swap*

Entrée: Graphe G (non orienté, liste d'adjacence)

Deux arêtes à échanger : (a, b) et (c, d)

Sortie : Mise à jour de $G.triangles$

```
1  $T \leftarrow G.triangles$ ;  
2 foreach  $x \in \{a, b\}$  do  
3   foreach  $y$  voisin de  $x$  où  $y \neq b$  et  $y \neq d$  do  
4     if  $(y, b)$  existe then  
5        $T \leftarrow T - 1$ ;  
6     end  
7     if  $(y, d)$  existe then  
8        $T \leftarrow T + 1$ ;  
9     end  
10   end  
11 end  
12 foreach  $x \in \{c, d\}$  do  
13   foreach  $y$  voisin de  $x$  où  $y \neq a$  et  $y \neq c$  do  
14     if  $(y, a)$  existe then  
15        $T \leftarrow T - 1$ ;  
16     end  
17     if  $(y, c)$  existe then  
18        $T \leftarrow T + 1$ ;  
19     end  
20   end  
21 end  
22  $G.triangles \leftarrow T$ ;
```
